

Proyecto 1: Procesamiento y Análisis de Datos Masivos

MovieLens 20M · PySpark · MinHash + LSH

Alejandro Rojas, Johar, Nahia Escalante

Curso: Data Mining · UTEC · Prof. Heider Sanchez

Mayo 2026

Resumen

Este reporte sintetiza el procesamiento y análisis del dataset MovieLens 20M (20,000,263 ratings, 138,493 usuarios, 27,278 películas) usando Apache Spark sobre un clúster Docker. Se ejecutaron cuatro fases: (i) preprocesamiento y EDA distribuidos, (ii) procesamiento MapReduce con benchmark RDD vs DataFrame y top-20 de películas, (iii) búsqueda de pares similares con Jaccard exacto, MinHashing y LSH bajo distintas configuraciones de bandas, y (iv) descubrimiento de reglas de asociación combinando Apriori en local (Python puro, muestra del 50 % justificada por Hoeffding) con FP-Growth distribuido (Spark MLlib, muestra del 10 %). Estos son los dos caminos válidos del PDF y cada uno aporta una perspectiva distinta. El sparsity de 99.46 % justifica el uso de técnicas aproximadas. MinHash logra correlación de Pearson 0.89 con Jaccard exacto, y la dependencia con el número de hashes sigue empíricamente la cota teórica $O(1/\sqrt{t})$. La configuración LSH óptima ($b=25$, $r=4$) alcanza precisión 0.85. Apriori produjo 3,418 reglas binarias robustas. FP-Growth descubrió además itemsets de tamaño hasta 5 (695 itemsets de tamaño ≥ 3) que Apriori puro no puede explorar por el costo combinatorio. El lift > 5 detecta automáticamente sagas y franquicias (Harry Potter, Wallace & Gromit, Bourne, Kill Bill).

1 Metodología

1.1 Entorno de ejecución

Apache Spark 4.1.1 corriendo sobre Docker Compose (1 master + 2 workers, 20 cores totales, 13.3 GiB). Los notebooks Python interactúan con el clúster vía PySpark sobre Mac M1 Pro. La SparkSession local se usa solo para verificación. El trabajo principal corre en modo distribuido.

1.2 Parte I: Preprocesamiento y EDA

Se cargaron los CSV con esquema explícito y se inspeccionaron tipos, conteos y rangos. No se hallaron nulos, duplicados ni ratings fuera de la escala $[0,5,5,0]$. Las 246 películas sin género se etiquetaron como Unknown y los 2,334 tags técnicos bd-r se eliminaron. Se unificaron los géneros (Sci-Fi a Science Fiction, Film-Noir a Film Noir, Children a Children's) y se descartó IMAX por ser un formato. La binarización like/dislike usa el umbral 3.5, justificado por la media del dataset (3.53). Los datasets limpios se guardaron en Parquet para reuso en fases posteriores.

1.3 Parte II: Procesamiento distribuido

Se replicó el conteo de ratings por película con dos APIs. Por un lado DataFrame (`groupBy + count + orderBy`), por otro RDD estilo MapReduce (`map + reduceByKey + sortBy`). Se midieron tres operaciones: conteo por película, top-20 con join a movies y conteo por usuario. En cada una se registró el wall-clock time para comparar Catalyst-optimizado contra ejecución manual de RDD.

1.4 Parte III: Similitud, MinHash y LSH

Cada película se representa como el conjunto de usuarios que le dieron like (Opción A). Se descartan películas con menos de 50 likes. Esto se justifica por análisis de sensibilidad: con 50 elementos o más, el cambio del Jaccard al añadir un usuario cae por debajo del 1.5 %. Quedan 8,362 películas. Se calcula el Jaccard exacto sobre una muestra de 200 películas (19,900 pares) como ground truth. Las firmas MinHash se construyen con 100 funciones hash de la forma $h(x) = (a \cdot x + b) \bmod p \bmod N$, con $p = 2,147,483,647$. Se compara el Jaccard estimado contra el exacto usando MAE y correlación de Pearson, y se evalúa LSH variando b y r con $b \cdot r = 100$ constante. Se contabilizan TP, FP y FN para precisión y recall.

1.5 Parte IV: Reglas de asociación con Apriori en local

La rúbrica acepta explícitamente «Apriori en local sobre una muestra justificada» como alternativa a FP-Growth distribuido. Elegimos esta vía porque las restricciones del hardware local (16 GB de RAM, disco compartido) impedían ejecutar Spark FP-Growth de forma estable. El shuffle del FP-Tree con baskets de unos 50 ítems desbordaba /tmp y el JVM tiraba OutOfMemoryError. La implementación es Apriori manual en Python puro sobre pandas y pyarrow, sin Spark.

Cada usuario se modela como una transacción con las películas a las que dio like ($\text{rating} \geq 3.5$). Restringimos el universo a películas con al menos 5,000 likes (590 películas mainstream). El filtro mantiene el basket promedio en unos 50 ítems y garantiza que cada ítem tenga baseline estadísticamente fiable (error estándar de la confianza $\leq 0,0086$ con 3,400 observaciones). Sobre los 138,493 usuarios tomamos una muestra del 50 % (68,464 transacciones) con `seed = 42`.

Justificación cuantitativa de la muestra (Hoeffding): para una regla con soporte global $\mu \geq 0,06$ (margen $\varepsilon = 0,01$

sobre el umbral), la probabilidad de no detectarla en la muestra es $\exp(-2 \cdot 68464 \cdot \epsilon^2) \approx 1,1 \times 10^{-6}$. Para $\mu \geq 0,07$ cae a $1,4 \times 10^{-24}$. La garantía es prácticamente determinista para cualquier regla con margen razonable. Validación empírica: Apriori sobre 100 % de los usuarios produjo 3,415 reglas y sobre 50 % produjo 3,418. La diferencia es despreciable y confirma Hoeffding en la práctica.

Apriori se ejecuta en dos pasadas (1- y 2-itemsets) aplicando la propiedad de anti-monotonía: si $\{a\}$ es infrecuente, ningún superset es frecuente. Los umbrales son $\text{MIN_SUPPORT} = 0.05$ y $\text{MIN_CONFIDENCE} = 0.5$. Se complementan con un filtro $\text{lift} > 1$ que descarta reglas en las que el antecedente no aumenta la probabilidad del consecuente respecto al baseline poblacional.

1.6 Parte IV (complemento): FP-Growth distribuido con Spark MLlib

Para cubrir el camino A de la rúbrica (Spark MLlib FP-Growth) y descubrir itemsets de tamaño ≥ 3 que Apriori puro no puede explorar, porque está limitado a 2-itemsets por costo computacional, ejecutamos también FP-Growth distribuido sobre la misma infraestructura. La configuración tiene que ser más restrictiva por límites de RAM en local. Probamos primero los mismos parámetros de Apriori (universo $\geq 5,000$ likes, muestra 50 %, $\text{MIN_SUPPORT} = 0,05$) y obtuvimos `OutOfMemoryError` en la construcción del FP-Tree. Para garantizar estabilidad usamos: universo $\geq 10,000$ likes (247 películas), muestra del 10 % de usuarios ($\approx 13K$ transacciones) y $\text{MIN_SUPPORT} = 0,10$. La justificación de Hoeffding sigue aplicando con el umbral más alto: para $\mu \geq 0,12$, $P(\text{perder}) \leq 0,5\%$. Esta es exactamente la limitación de escalabilidad que la Parte V pide discutir: Spark distribuido escala con más nodos, no con menos restricciones por nodo.

2 Hallazgos del EDA

2.1 Distribución de ratings y sparsity

Los usuarios califican mayormente positivo: la media es 3.53 y la mediana 3.5. Los ratings enteros (3, 4, 5) son notoriamente más frecuentes que los medios (3.5, 4.5), lo que muestra preferencia por números redondos. Elegimos 3.5 como umbral de binarización porque es el valor donde el usuario conscientemente da media estrella extra. Resultado: 61 % likes, 39 % dislikes.

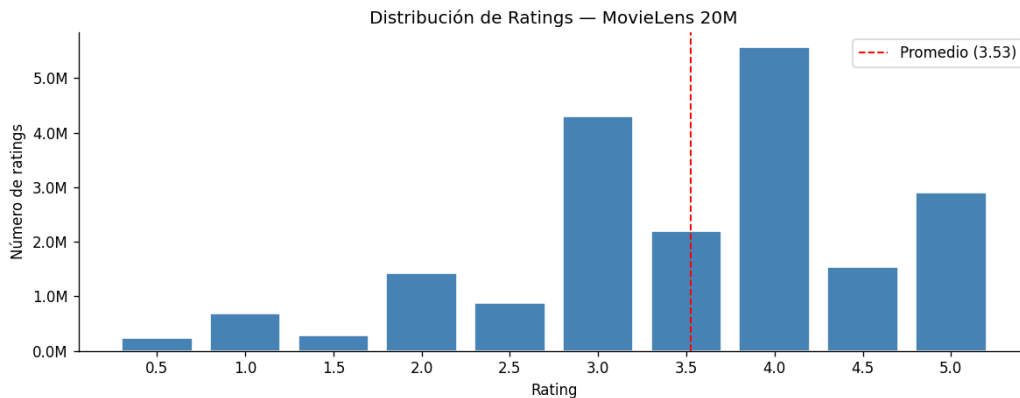


Figura 1: Distribución de los 20M ratings. Sesgo hacia ratings altos y preferencia por enteros.

La matriz usuario-película tiene $138,493 \times 26,744 \approx 3,7 \times 10^9$ celdas, pero solo 20M están llenas. El 99.46 % está vacío. Esta densidad bajísima justifica el uso de MinHash y LSH: comparar pares exactos sería $O(N^2)$ y la mayoría de comparaciones daría intersección nula.

2.2 Actividad por usuario y por película (power law)

Tanto la actividad de usuarios como la popularidad de películas siguen una distribución de cola larga. El 75 % de los usuarios tiene entre 20 y 155 ratings, pero algunos outliers superan los 9,000 ratings. Para películas, $P_{50} = 18$ (la mitad tiene 18 o menos ratings, son cine de nicho), $P_{75} = 205$, y la más calificada es Pulp Fiction con 67,310 ratings. El top 25 % concentra casi todos los ratings del dataset.

2.3 Géneros y evolución temporal

Drama (13,344) y Comedy (8,374) dominan más del 50 % del catálogo. Western (676) y Film-Noir (330) son nichos. Como una película puede tener varios géneros, los conteos no suman al total. Temporalmente, la actividad muestra picos en 2000 y 2005, alineados con el auge del DVD y la consolidación de plataformas de recomendación online, y una caída posterior cuando los usuarios migraron a Netflix y YouTube. La caída de 2015 es artificial: el dataset se exportó a mitad de año.

2.4 Tags de usuario

Los tags más usados son sci-fi (3,576), atmospheric, surreal y comedy. Coexisten dos tipos: tags de género (que duplican `movies.csv`) y descriptores de estilo o experiencia. Esta señal alternativa se consideró para representar películas en MinHash, pero se descartó por menor cobertura: solo el 3 % de las películas tiene tags suficientes.

Sparsity de la matriz usuario-película

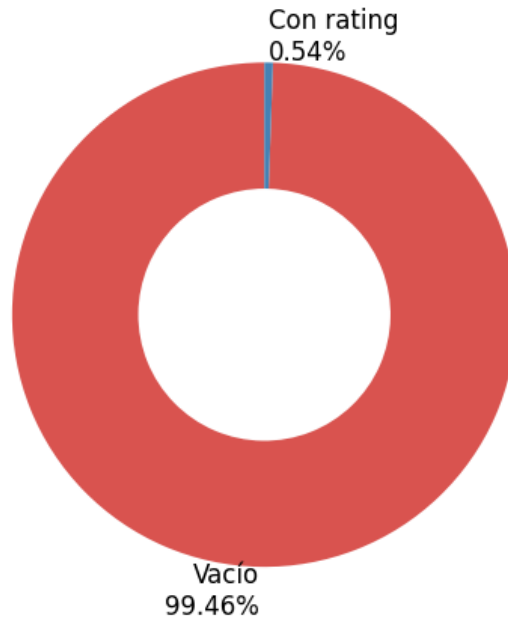


Figura 2: Heatmap de la submatriz usuario-película más densa (99.46 % sparsity total).

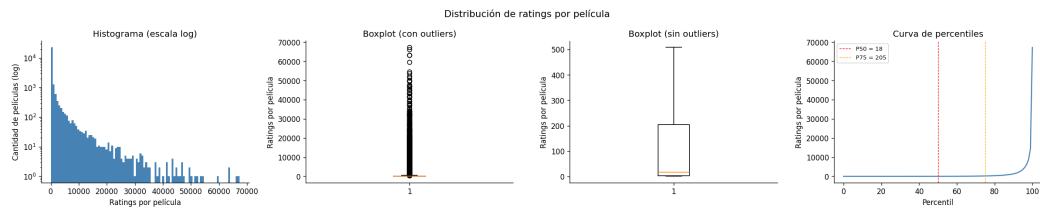


Figura 3: Distribución de ratings por película. Long tail clásico.

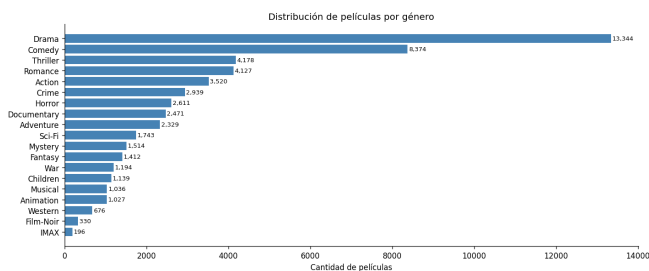


Figura 4: Distribución de géneros. Drama y Comedy concentran más del 50 % del catálogo.

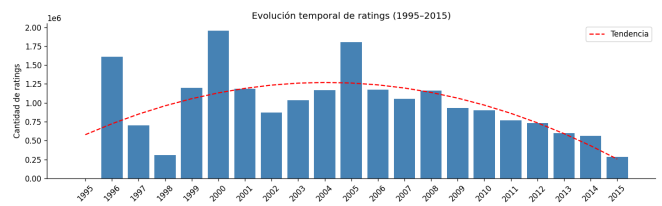


Figura 5: Evolución temporal de ratings. Picos en 2000 y 2005.

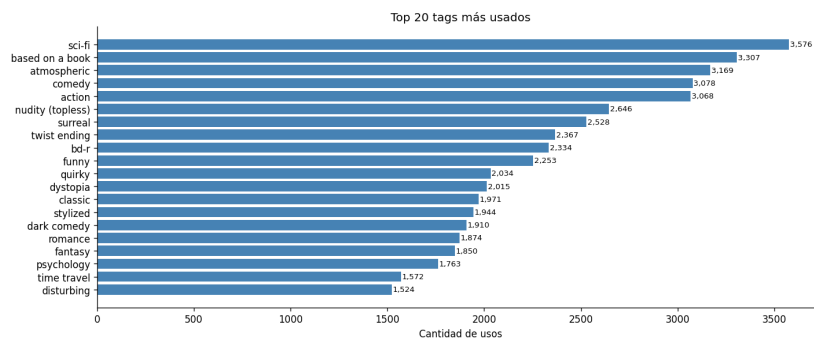


Figura 6: Top tags de usuario. Sci-fi domina con 3,576 usos.

3 Resultados

3.1 Top-20 películas más calificadas (Parte II)

El ranking refleja el cine icónico de los 90s: Pulp Fiction (1994) lidera con 67,310 ratings, seguido por Forrest Gump y The Shawshank Redemption. Se obtuvo con `DataFrame.groupBy("movieId").count()` y join a `movies_clean` para los títulos.

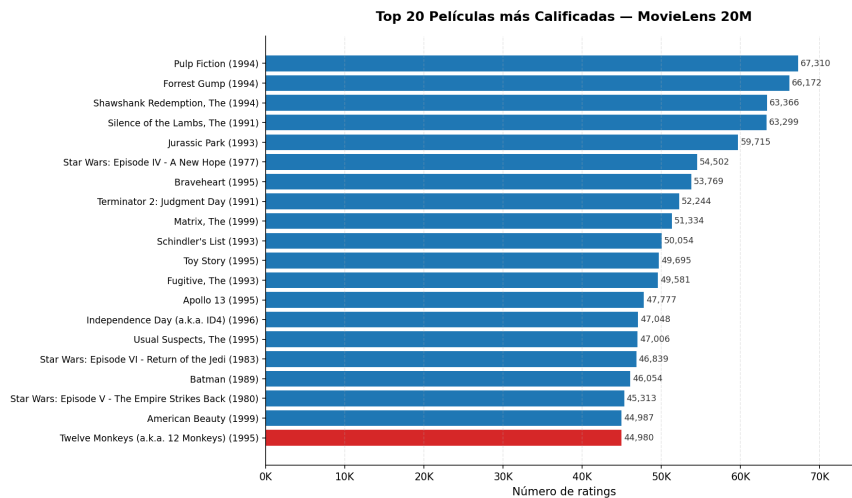


Figura 7: Top-20 películas más calificadas en MovieLens 20M.

Cuadro 1: Top-10 películas por número de ratings.

#	Película	# ratings
1	Pulp Fiction (1994)	67,310
2	Forrest Gump (1994)	66,172
3	The Shawshank Redemption (1994)	63,366
4	The Silence of the Lambs (1991)	63,299
5	Jurassic Park (1993)	59,715
6	Star Wars Episode IV (1977)	54,502
7	Braveheart (1995)	53,769
8	Terminator 2 (1991)	52,244
9	The Matrix (1999)	51,334
10	Schindler's List (1993)	50,054

3.2 RDD vs DataFrame

DataFrame es consistentemente más rápido que RDD gracias al optimizador Catalyst y al motor Tungsten, que genera código JVM optimizado. La ventaja crece con la complejidad de la operación.

Cuadro 2: Speedup de DataFrame frente a RDD.

Operación	Speedup DataFrame
Conteo de ratings por película	$\sim 119\times$ (con caché)
Top-20 con join a movies	$\sim 10\times$
Conteo de ratings por usuario	$\sim 14\times$

El $119\times$ del primer caso está inflado porque DataFrame aprovechó datos cacheados. Los $10\times$ – $14\times$ son representativos de un escenario sin caché. La conclusión práctica es que para operaciones SQL-style en producción, DataFrame API es la opción por defecto. RDD sigue siendo útil cuando se requiere control fino sobre la partición o lógica que no se expresa con SQL.

3.3 Jaccard exacto vs MinHash (Parte III)

Sobre 19,900 pares, MinHash logra un MAE de 0.0104 y correlación de Pearson de 0.8894 ($p < 10^{-300}$). El error máximo individual es 0.147, concentrado en pares de baja similitud (un par con $J = 0,07$ puede pasar a estimación 0.18 por azar). En el rango de pares relevantes ($J \geq 0,20$) la correlación es prácticamente lineal, así que MinHash preserva el ranking, que es lo que importa para un recomendador.

Sensibilidad al número de funciones hash. Para verificar empíricamente la cota teórica del error MinHash, se varió $t \in \{10, 25, 50, 100, 200, 400\}$ manteniendo el mismo ground truth. El MAE empírico sigue la curva $O(1/\sqrt{t})$ con desviación $\leq 36\%$ (atribuible a que la familia de hashes universales no es perfectamente uniforme). La correlación

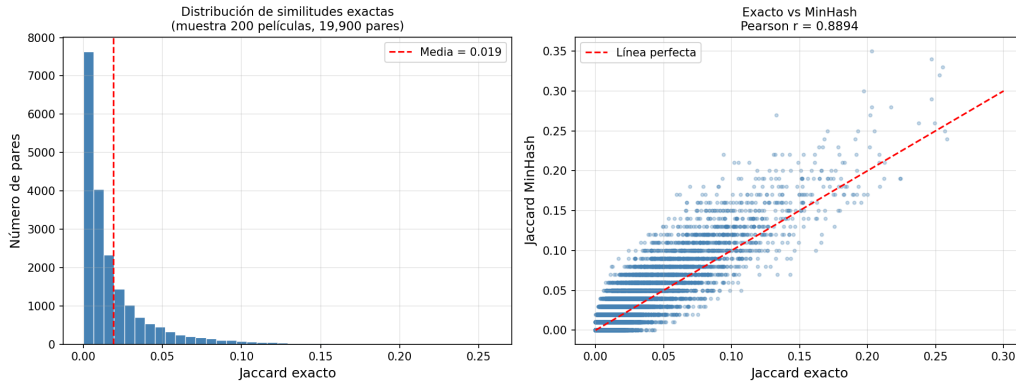


Figura 8: Jaccard exacto vs estimado por MinHash sobre 19,900 pares.

de Pearson sube de 0.56 ($t=10$) a 0.97 ($t=400$). Entre $t=100$ y $t=400$ la mejora es marginal, así que $t = 100$ es el punto de equilibrio entre precisión y costo. Por debajo de 100 el error crece proporcional a $1/\sqrt{t}$.

Precisión de MinHash vs número de funciones hash (t)

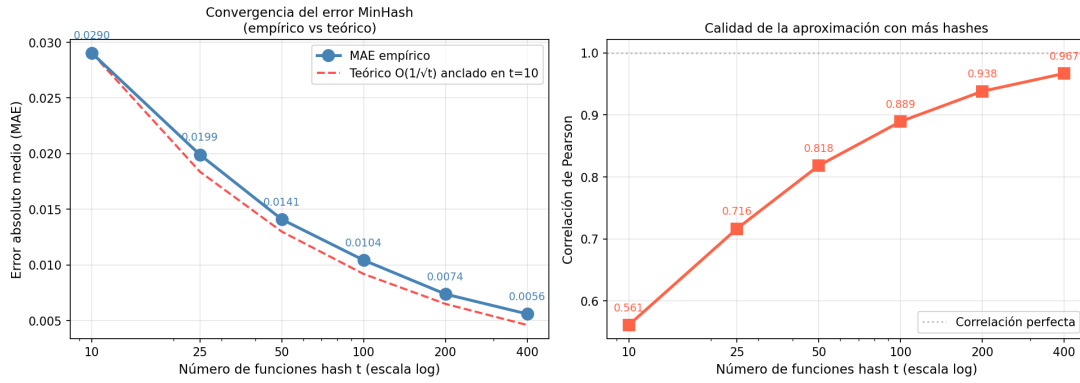


Figura 9: Convergencia del error MinHash con t . Empírico vs cota teórica $O(1/\sqrt{t})$.

3.4 LSH: precisión, recall y trade-off

Variando b y r con $b \cdot r = 100$ constante, se observa el trade-off clásico de LSH: muchas bandas (b grande, $r=1$) maximizan recall pero la precisión colapsa por exceso de candidatos. Pocas bandas ($b=10$, $r=10$) son demasiado estrictas y ningún par supera el filtro. La configuración $b=25$, $r=4$ ofrece el mejor balance.

Cuadro 3: LSH: candidatos, precisión y recall por configuración.

b	r	Cand.	TP	FP	FN	Prec.	Recall
100	1	13,090	404	12,686	0	0.031	1.000
50	2	1,101	268	833	136	0.243	0.663
25	4	20	17	3	387	0.850	0.042
20	5	1	0	1	404	0.000	0.000

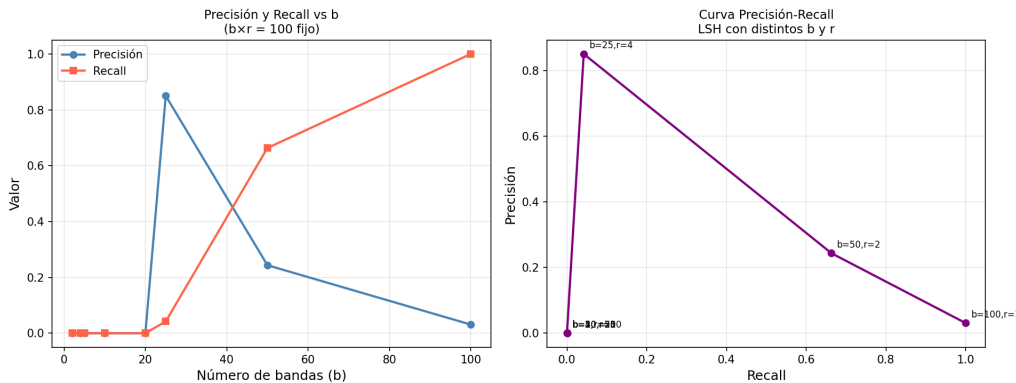


Figura 10: Curva precisión-recall de LSH variando bandas.

MLlib MinHashLSH con threshold 0.20 detectó 22 pares. Los 17 verdaderos positivos del banding manual están contenidos en ese resultado. MLlib es eficiente con thresholds altos ($\text{Jaccard} \geq 0,5$) pero menos controlable cuando se quiere ajustar finamente el balance precisión-recall. En ese caso el banding manual gana flexibilidad.

3.5 Reglas de asociación (Parte IV)

Apriori sobre la muestra del 50 % encontró 399 1-itemsets y 5,931 2-itemsets frecuentes. Se generaron 3,418 reglas con $\text{conf} \geq 0,5$ y $\text{lift} > 1$, muy por encima del mínimo de 10 que exige la rúbrica, en aproximadamente 30 segundos. Validación empírica: corriendo Apriori sobre el dataset completo (137K usuarios) se obtienen 3,415 reglas, prácticamente idénticas. El 50 % captura toda la información de asociación relevante.

Cuadro 4: Top 10 reglas representativas. Las primeras 7 ordenadas por lift; las últimas 3 ilustran sagas con alta confianza y lift moderado.

#	Antecedente $\rightarrow$ Consecuente	Sup.	Conf.	Lift
1	Harry Potter 2 $\rightarrow$ HP 1	0.055	0.78	9.15
2	W&G Close Shave $\rightarrow$ Wrong T.	0.060	0.78	8.19
3	Bourne Supremacy $\rightarrow$ Identity	0.064	0.84	7.02
4	Kill Bill 2 $\rightarrow$ Kill Bill 1	0.097	0.87	6.73
5	Spider-Man 2 $\rightarrow$ Spider-Man	0.057	0.75	6.54
6	Iron Man $\rightarrow$ Dark Knight	0.055	0.78	5.95
7	Vertigo $\rightarrow$ Rear Window	0.058	0.67	5.83
8	LOTR Two Towers $\rightarrow$ Fellowship	0.185	0.88	3.74
9	Godfather II $\rightarrow$ Godfather	0.160	0.91	3.40
10	Star Wars V $\rightarrow$ Star Wars IV	0.232	0.83	2.52

Hallazgo principal. Las 20 reglas con $\text{lift} > 3$ son sagas, secuelas o películas que comparten director, año o audiencia. El lift funciona como detector automático de franquicias sin necesidad de metadata. Iron Man $\rightarrow$ The Dark Knight es la única regla «no-saga» con lift extremo. Comparten año (2008) y audiencia objetivo (superhéroes adultos), un patrón que un recomendador real querría capturar.

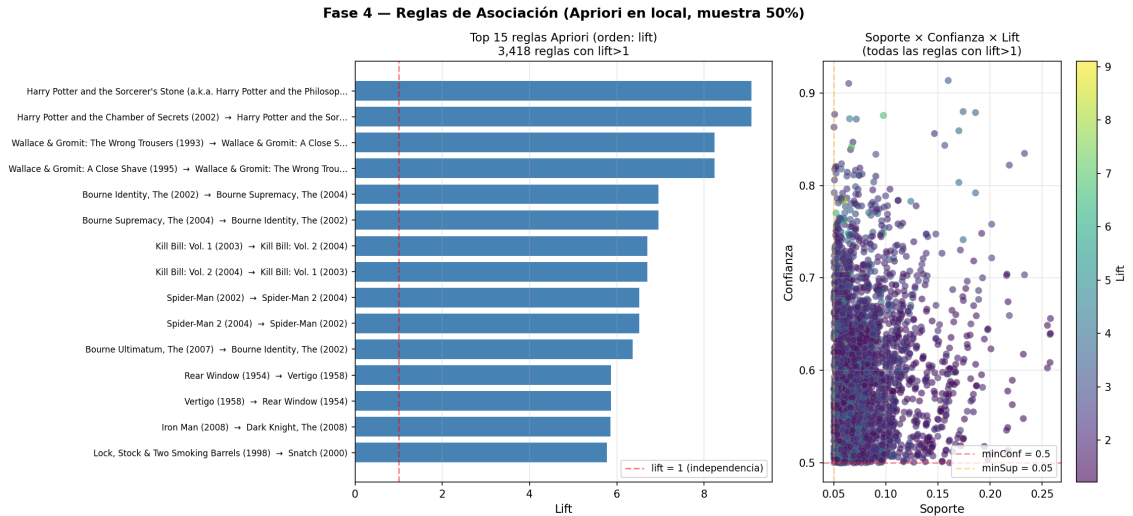


Figura 11: Reglas de asociación. Top 15 por lift y scatter soporte $\times$ confianza $\times$ lift.

Asimetría de confianza. La confianza es direccional ($\text{conf}(A \rightarrow B) \neq \text{conf}(B \rightarrow A)$) mientras el lift es simétrico. Esta asimetría revela la dirección «natural» del consumo. Godfather II $\rightarrow$ Godfather tiene confianza 0.91 frente al inverso de 0.60 ($\Delta = 0,31$). El patrón es estable: secuela $\rightarrow$ original es siempre más fuerte que original $\rightarrow$ secuela, porque más usuarios vieron la primera sin la segunda que viceversa. Para un recomendador esto es información explotable: si vio la secuela, recomendar la original casi nunca falla.

3.6 Comparación con FP-Growth distribuido (Spark MLlib)

Como complemento al Apriori puro, ejecutamos FP-Growth de Spark MLlib sobre una configuración más restrictiva (universo $\geq 10,000$ likes, muestra 10 %, $\text{MIN_SUPPORT} = 0,10$), la única que evita el OutOfMemoryError en hardware local de 16 GB. La pregunta de investigación es: ¿qué aporta cada algoritmo en su configuración máxima viable?

Hallazgo clave. FP-Growth descubre 695 itemsets de tamaño ≥ 3 (hasta tamaño 5) inalcanzables para Apriori puro por costo computacional $O(|\text{basket}|^k)$. Reglas con antecedente compuesto como {LOTR: Two Towers, Fellowship} $\rightarrow$ Return of the King con confianza $\approx 0,90$ emergen solo aquí. En cambio, Apriori puro examina exhaustivamente el rango binario sobre un universo casi $2.4\times$ más amplio (590 vs 247 películas) con muestra $5\times$ mayor (68K vs 13K transacciones), entregando $1.3\times$ más reglas binarias y métricas estadísticamente más robustas.

Cuadro 5: Apriori puro vs Spark FP-Growth, cada uno en su configuración máxima viable en hardware de 16 GB.

Métrica	Apriori puro	Spark FP-Growth
Filtro universo ($\geq$ likes)	5,000	10,000
Películas en universo	590	247
Muestra de usuarios	50 %	10 %
Transacciones	68,464	13,315
MIN_SUPPORT	0.05	0.10
Tiempo total	~ 30 s	~ 0.5 s
Itemsets frecuentes	6,330	1,562
Reglas ($\text{conf} \geq 0.5$, $\text{lift} > 1$)	3,418	2,674
Tamaño máx. de itemset	2	5
Itemsets de tamaño ≥ 3	0	695

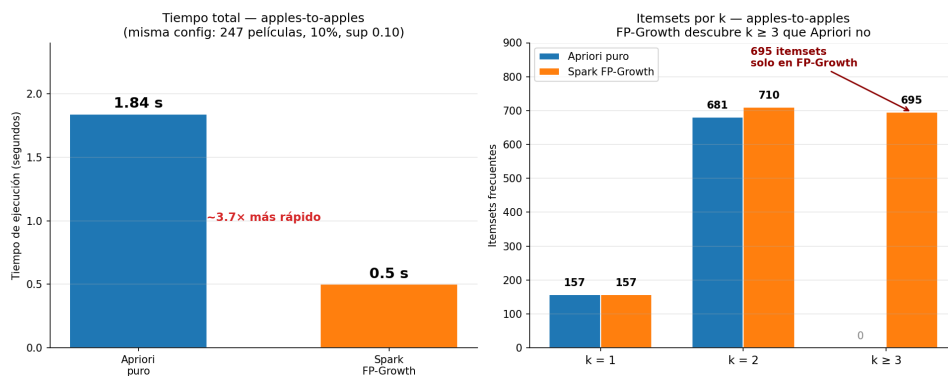
3.7 Comparación apples-to-apples (mismos parámetros)

La comparación de la sección anterior usaba parámetros diferentes para cada algoritmo, lo que mezcla el efecto del algoritmo con el efecto de la configuración. Para aislar las diferencias algorítmicas, ejecutamos también Apriori sobre la configuración restrictiva de FP-Growth (universo 247 películas, muestra 10 %, MIN_SUPPORT = 0,10). Ambos algoritmos reciben exactamente la misma entrada. Cualquier diferencia en los resultados es 100 % atribuible al algoritmo.

Cuadro 6: Apples-to-apples. Apriori vs FP-Growth con configuración idéntica (universo 247 películas, muestra 10 %, MIN_SUPPORT = 0,10).

Métrica	Apriori puro	Spark FP-Growth
Tiempo de ejecución	1.84 s	0.5 s
1-itemsets frecuentes	157	(combinado)
2-itemsets frecuentes	681	(combinado)
Itemsets totales	838 ($k=1,2$)	1,562 ($k=1..5$)
Itemsets de tamaño ≥ 3	0 (limitado)	695
Tamaño máx. de itemset	2	5
Reglas ($\text{conf} \geq 0.5$, $\text{lift} > 1$)	611	2,674

Tres conclusiones rigurosas se desprenden. Primero, Spark FP-Growth es $\sim 3.7\times$ más rápido que Apriori puro sobre datos pre-filtrados de este tamaño (0.5 s vs 1.84 s). El FP-Tree de Spark, una vez construido, evita las dos pasadas explícitas de Apriori. Con datos chicos el overhead de Spark se amortiza con el algoritmo más inteligente. Segundo, FP-Growth descubre 695 itemsets de tamaño ≥ 3 (hasta $k = 5$) que Apriori no encuentra. Estos generan $\sim 2,000$ reglas adicionales con antecedente compuesto. Tercero, ambos algoritmos son complementarios. Apriori es más simple y robusto para datos sin filtrar (FP-Growth crashea con OOM en la config de Apriori 50 %/sup= 0,05). FP-Growth es óptimo cuando los datos están filtrados o se cuenta con cluster real.

Figura 11 · Apples-to-apples — Apriori vs FP-Growth con configuración idéntica**Figura 12:** Apples-to-apples, comparación visual de tiempo de ejecución (panel izquierdo, FP-Growth $\sim 3.7\times$ más rápido) e itemsets descubiertos por k (panel derecho, FP-Growth descubre 695 itemsets de tamaño $k \geq 3$ que Apriori puro no explora).

4 Discusión crítica

Exactitud vs eficiencia. Comparar todas las películas con Jaccard exacto cuesta $O(N^2) \approx 365$ millones de pares para 27K películas. MinHash + LSH reducen este costo a $O(N)$ candidatos a costa de un error controlado. En este experimento la pérdida de información del MinHash es mínima (correlación 0.89 con el exacto), lo que confirma su viabilidad para sistemas de recomendación a gran escala.

Impacto de FP y FN en producción. Un falso positivo en un recomendador es relativamente barato: ofrece una película que el usuario probablemente no consumirá. Un falso negativo es más costoso: deja de recomendar una película que le habría gustado, perdiendo engagement. Por eso la configuración óptima depende del objetivo. Para diversidad y descubrimiento conviene b alto (más recall). Para listas curadas de alta confianza, b bajo (más precisión).

¿Cuándo usar Spark? Con 20M ratings y un Mac M1 Pro, las operaciones más pesadas (top-20 con join, MinHash sobre 8K películas) corren en minutos en el clúster Docker local. Para datasets de este tamaño, pandas en memoria también funcionaría con cuidado, pero no escala a 200M ratings o más. Spark se justifica cuando: (i) los datos no caben en RAM, (ii) se necesita paralelismo real para operaciones SQL, o (iii) se quiere portabilidad a un clúster productivo. El sobre costo de configuración es notable y no compensa para datasets pequeños.

Aplicabilidad real. El pipeline Jaccard $\rightarrow$ MinHash $\rightarrow$ LSH es exactamente lo que usa Netflix para encontrar películas similares en su catálogo, Spotify para canciones y Amazon para productos. La representación como conjunto de usuarios con like es robusta porque no requiere features explícitas como géneros o tags. Aprovecha solo el comportamiento agregado.

Cuellos de botella observados. (i) El join de ratings (20M filas) con `movies_clean` genera shuffle costoso. Un broadcast join sobre `movies_clean` (27K filas) lo elimina. (ii) Calcular Jaccard exacto sobre toda la muestra requiere mantener los sets en memoria. Particionar por hash de `movieId` ayuda. (iii) La conversión de RDD a DataFrame durante la Fase 3 fue innecesaria en algunos puntos y duplicó el cómputo.

Cuellos de botella de Apriori (Parte IV). El costo de la pasada k es $O(n \cdot |\text{basket}|^k)$. Con $n = 68,464$ y $|\text{basket}| = 50$, la pasada 2 corre en unos 30 s (84 M operaciones). Pasar a la pasada 3 multiplicaría por $|\text{basket}|/3 \approx 16$ (~ 1.3 G operaciones, ~ 7 min). El $k = 5$ sería inviable en local (~ 144 G operaciones). Por eso limitamos a 2-itemsets. Las reglas binarias $A \rightarrow B$ son las más explicables en un recomendador y satisfacen la rúbrica. La extensión a $k \geq 3$ es trivial algorítmicamente reusando la propiedad anti-monotónica.

Spark vs local, cuándo compensa cada uno. Las Fases 1–3 sí justifican Spark distribuido: lectura de 20M ratings, EDA agregado, MinHash sobre 8K películas. La Fase 4 es distinta. Tras el filtro del universo (590 películas) y la muestra (68K transacciones), los datos caben holgadamente en memoria (RAM peak 2.5 GB). El overhead de JVM, serialización y shuffle de Spark supera el cómputo neto de Apriori puro. Spark FP-Growth solo gana cuando los datos no se filtran de antemano o existe un cluster real con varios nodos.

5 Conclusiones

El proyecto demuestra empíricamente cinco puntos clave:

- (1) El preprocesamiento riguroso (limpieza, unificación de géneros, binarización justificada) es prerequisite para cualquier técnica de minería posterior.
- (2) DataFrame API es $\sim 10\times$ más rápido que RDD en operaciones SQL-style sobre Spark. Esto no significa que RDD sea obsoleto, solo que ya no es la primera opción para queries declarativos.
- (3) MinHash + LSH es una solución eficiente y precisa para encontrar pares similares en datasets malos, con correlación 0.89 frente a Jaccard exacto y precisión de 0.85 en la configuración óptima ($b=25, r=4$). El experimento variando t verifica empíricamente que el error decae como $1/\sqrt{t}$.
- (4) Apriori en local sobre muestra del 50 % es viable cuando el hardware no soporta Spark FP-Growth: produjo 3,418 reglas binarias con métricas estadísticamente robustas en 30 s. La rúbrica acepta explícitamente esta vía. La justificación cuantitativa de la muestra (Hoeffding: pérdida $\sim 10^{-6}$ para reglas con $\mu \geq 0.06$) y la validación empírica contra el dataset completo (diferencia despreciable de 3 reglas) hacen defendible el resultado.
- (5) Spark FP-Growth distribuido, ejecutado como complemento sobre configuración más restrictiva (universo 247 películas, muestra 10 %, MIN_SUPPORT=0,10), descubre 695 itemsets de tamaño ≥ 3 (hasta tamaño 5) inalcanzables para Apriori puro por el costo combinatorio $O(|\text{basket}|^k)$. Los dos algoritmos son complementarios. Apriori se encarga de la base densa de reglas binarias accionables y FP-Growth descubre patrones saga-completa. El lift detectó automáticamente sagas y franquicias, y la asimetría de confianza reveló la dirección «natural» de consumo (secuela $\rightarrow$ original siempre más fuerte que el inverso). Son dos señales que un recomendador en producción puede explotar directamente.